

オンライン対戦SRPG 仕様書 (v0.1)

Godot 4.x + PartyKit / ファミコンウォーズ風 1on1 SRPG

0. このプロジェクトの目的

「マルチプレイヤーのゲームを成立させること」が最優先。ゲームデザインの新規性は二の次で、ネットワーク同期・マッチング・切断復帰・権威サーバーの扱いを実際に手を動かして解くのが本題。

- 販売なし / ポートフォリオ + ブログ記事用
- ソロプレイ、キャンペーン、CO能力、実績などは**スコープ外**
- 「動くものを最短で出して、そこから面白さを足す」方針

非目標（やらないと明言しておくリスト）

やらないことを最初を書いておかないと必ず膨らむので明記する。

- ソロモード / AI対戦
- マップエディタ
- ランクマッチ、レーティング、永続アカウント
- ボイスチャット、自由入力チャット
- モバイル対応（画面設計はPC前提）

1. 技術構成

レイヤ	採用	補足
クライアント	Godot 4.x (GDScript)	Web export (HTML5) を第一ターゲット
通信	WebSocket (WebSocketPeer)	Godot標準の高レベルマルチプレイAPIは使わない
サーバー	PartyKit (TypeScript / Cloudflare Durable Objects)	1マッチ = 1 Room = 1 Durable Object
ホスティング	Cloudflare (PartyKit) + 静的配信	ブログから <iframe> で埋め込み想定

なぜ Godot の MultiplayerAPI を使わないか

Godotの ENetMultiplayerPeer / MultiplayerSynchronizer はP2P・ホスト権威のリアルタイムゲーム向けで、PartyKitのような**外部のWebSocketサーバーが権威を持つ構成**とは噛み合わない。ターン制なら同期頻度が低いので、WebSocketPeer に自前のJSONプロトコルを乗せるほうが素直かつデバッグしやすい。

PartyKit を選ぶ理由

- Room単位で状態を持つDurable Objectのモデルが「1試合 = 1部屋」に完全に一致する
- WebSocketの接続管理・ブロードキャストが最初から用意されている
- サーバーレスなので待機中のコストがほぼゼロ（ポートフォリオ向きの重要な性質）
- 記事にしたとき「なぜこの構成か」が説明しやすい

2. ネットワーク設計（ここが本題）

2.1 権威モデルの選択

3案あり、**案B** を採用する。

案	内容	評価
A: 純リレー	サーバーは中継のみ、両クライアントが決定論的に同じ計算	実装最小だが、クライアント改造でチートし放題。デシンク時に検知不能
B: 検証付きリレー	サーバーは ルールの一部だけ 検証して中継。乱数はサーバーが振る	実装量とのバランスが最良。 採用
C: 完全権威	サーバーが全ゲームロジックを保持	ルールをGDScriptとTSで二重実装になる。ポートフォリオには重すぎる

案Bでサーバーが検証すること

ゲームルール全部ではなく、「**不正の被害が大きく、かつ検証が安い**」ものだけを見る。

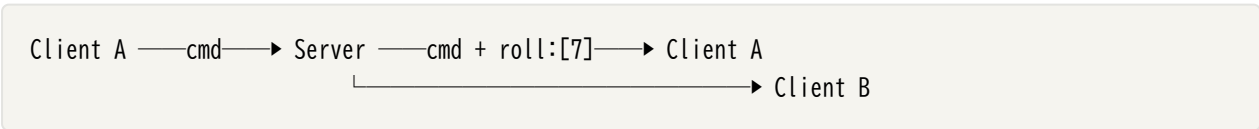
- 送信者は本当に現在の手番プレイヤーか
- 対象ユニットは送信者の所有物か
- そのユニットはこのターンまだ行動していないか
- コマンド種別・IDのスキーマが正しいか
- 資金が足りているか（生産コマンド時、サーバーも資金だけは持つ）

移動経路の妥当性や地形コストの計算は**クライアント側に任せる**。ここまでやると結局ルール全実装になるため。代わりに後述のチェックサム照合で異常を検知する。

乱数はサーバーが振る

ファミコンウォーズ系のダメージ計算には乱数（±10%程度の振れ幅）がある。これをクライアント側でシードPRNGを回して同期させると、乱数の呼び出し順が1回でもズレた瞬間にデシンクする。**呼び出し順の完全一致は想像以上に壊れやすい**。

そこでサーバーがコマンドをブロードキャストする際に、必要な乱数値を配列で添付する。クライアントは自分で乱数を振らず、渡された値を使う。これで決定論の要求が大幅に下がる。



2.2 コマンドログ方式

サーバーは試合中、**確定したコマンドの配列**を保持する。

```
type MatchState = {  
  seq: number;           // 単調増加のコマンド番号  
  log: Command[];        // 確定コマンド全履歴  
  turn: number;  
  activePlayer: 0 | 1;  
  funds: [number, number];  
  phase: "waiting" | "playing" | "ended";  
};
```

このログひとつで以下が全部ただで手に入る:

- **再接続復帰**: 切断したクライアントは resync を投げ、ログ全体を受け取って先頭から再生
- **観戦**: 途中参加者にログを流し込むだけ
- **リプレイ**: 試合終了時にログをJSONで吐けばそのまま完全再生できる
- **デシンク検知**: 各クライアントが定期的に状態ハッシュを送り、不一致を検出

コマンドログはゲーム状態そのものより圧倒的に小さい（1試合で数KB程度）ので、Durable Objectのストレージにもそのまま置ける。

2.3 プロトコル

JSON。{ "t": <type>, ... } 形式。

Client → Server

type	ペイロード	説明
hello	name	接続直後。マッチング待機列へ
ready	—	マップ受領完了、開始OK
cmd	cmd: Command	行動コマンド送信
resync	fromSeq	指定seq以降のログ要求
hash	seq, h	状態チェックサム報告
surrender	—	投了
pong	id	死活応答

Server → Client

type	ペイロード	説明
matched	roomId, playerId, opponentName	相手が見つかった
start	map, units, funds, firstPlayer	初期状態配布
commit	seq, cmd, rolls[]	コマンド確定。これを受けて初めて描画が動く
turn	turn, activePlayer, funds, deadline	手番交代
log	commands[]	resync応答
end	winner, reason	決着
desync	seq	状態不一致の通知（強制resync）
oppLeft	graceUntil	相手切断、猶予時間つき
ping	id	死活確認

Command の形

```
type Command =
  | { k: "move";    unit: number; path: number[]; then: "wait" }
  | { k: "attack";  unit: number; path: number[]; target: number }
  | { k: "capture"; unit: number; path: number[] }
  | { k: "build";   tile: number; unitType: string }
  | { k: "endTurn" };
```

座標は x, y ではなく**タイルindex** ($y * \text{width} + x$) の**整数1個**で持つ。JSONが軽くなり、比較・ハッシュ化も楽。

2.4 楽観的更新はしない

ターン制なので**しない**。ローカルで先に動かして後で巻き戻す複雑さに見合うメリットがない。

自分のコマンドも「サーバーの commit が返ってきてから描画する」。往復100ms程度なら体感で問題にならない。ただし**入力受付は即座にロックし、ボタンを押した瞬間にUI上のフィードバック(選択ハイライト等)は出すこと**。無反応に見えるのが一番ストレスになる。

2.5 切断とタイムアウト

事象	挙動
WebSocket切断	60秒の復帰猶予。相手には oppLeft で残り時間を表示
猶予内に再接続	resync でログ再生し復帰
猶予切れ	切断側の敗北で end
ターン時間切れ	サーバーが自動で endTurn を発行 (90秒)
3ターン連続で無操作	放置とみなし敗北扱い

ターン制限時間はサーバーが持つ (クライアントの時計は信用しない)。turn メッセージに deadline (サーバー時刻のUnix ms) を含め、クライアントはそれとの差分を表示するだけにする。

3. ゲームルール (v1)

原作準拠の最小セット。**まず動かす**ことを優先し、ユニット数も絞る。

3.1 基本

- 2人対戦、交互ターン制
- マップは矩形グリッド (v1は 15×10 固定の1マップのみ)
- 1ターンで自軍全ユニットを1回ずつ行動可能
- 行動 = 移動 → 「攻撃 / 占領 / 待機」のいずれか

3.2 ユニット (v1は6種のみ)

ユニット	コスト	移動	移動型	射程	弾	燃料	備考
歩兵	1000	3	歩行	1	∞	99	占領可
対戦車兵	3000	2	歩行	1	3	70	占領可 / 対車両特効
戦車	7000	6	キャタピラ	1	9	70	汎用
中戦車	16000	5	キャタピラ	1	8	50	高耐久
自走砲	6000	5	キャタピラ	2-3	9	50	間接攻撃・移動後攻撃不可
リコン	4000	8	タイヤ	1	∞	80	高機動・偵察

航空・海上ユニットは**v2以降**。移動型が増えると地形コスト表が膨らむため。

3.3 地形

地形	防御	歩行	キャタピラ	タイヤ
平地	1	1	1	1
森	2	1	2	3
山	4	2	－	－
道路	0	1	1	1
川	0	2	－	－
都市	3	1	1	1
工場	3	1	1	1
司令部	4	1	1	1

- は進入不可。

3.4 ダメージ計算

原作系の式を踏襲。HPは内部10、表示も10。

```
基礎 = 攻撃力表[攻撃側][防御側] // 0-100
攻撃補正 = 攻撃側HP / 10
防御補正 = (200 - 地形防御 * 10 * 防御側HP / 10) / 100
乱数 = サーバー生成の 0-9

ダメージ = floor( 基礎 * 攻撃補正 * 防御補正 / 10 ) + 乱数補正
```

すべて整数演算で行う。浮動小数点はクライアント間で誤差が出うるため、表示以外では一切使わない（v1は乱数がサーバー生成なので厳密には不要だが、チェックサム照合を成立させるために整数統一する）。

反撃: 隣接した直接攻撃で、防御側が生存かつ直接攻撃ユニットなら反撃発生。反撃時のHPは被弾後の値を使う。

3.5 経済と生産

- ターン開始時、占領済みの都市・工場・司令部1つにつき **1000** 加算
- 工場タイル上に自軍ユニットがない場合のみ生産可能
- 生産したユニットはそのターン行動不可

3.6 占領

- 歩兵・対戦車兵のみ可能
- 占領値20。1ターンにつき「自HP」分を減算（HP10なら2ターンで完了）
- 占領中に移動 or 被弾でHPが減ると、占領値は20にリセット

3.7 勝敗

- 相手司令部を占領 → 勝利
- 相手ユニット全滅 → 勝利
- 投了 → 相手の勝利
- 切断猶予切れ → 相手の勝利
- **30ターン経過** → 保有資産（占領施設数 + 全ユニット価値）が多いほうの勝利

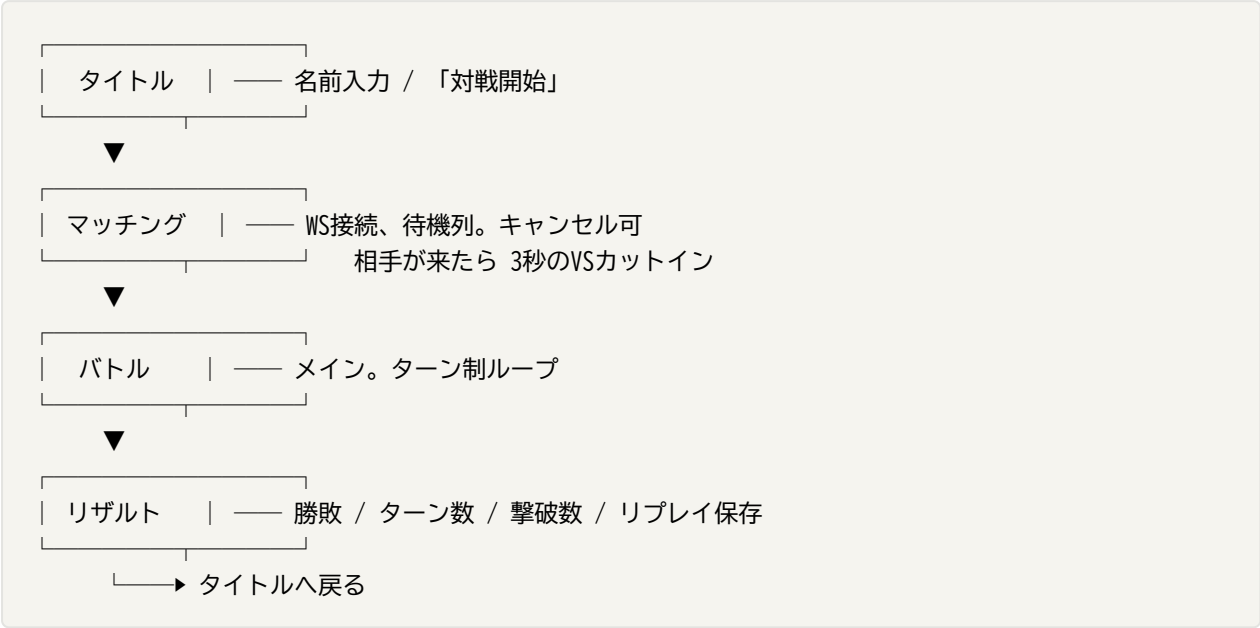
ターン上限は原作にないが、**オンライン対戦では必須**。決着がつかない試合はサーバーリソースを食い続けるし、ポートフォリオのデモとしても間延びする。

3.8 索敵（Fog of War）はv1では実装しない

理由: 隠蔽情報が入ると「クライアントに全状態を送れない」ため、案Bの検証付きリレーが破綻し、案C（完全権威）が必須になる。実装コストが跳ね上がるので、**v2以降の課題**として明示的に切る。

記事的にはむしろ「なぜFoWだけは権威サーバーが必須なのか」を書けるので、切ったこと自体がコンテンツになる。

4. 画面フロー



バトル画面の構成

- 中央: グリッドマップ（カメラはドラッグ / WASD で移動、ホイールでズーム）
- 上部: ターン数、手番表示、残り時間バー
- 左上: 自軍資金
- 右下: カーソル下タイルの地形情報 + ユニット詳細
- 下部: 行動メニュー（コンテキストメニュー方式）

入力

操作	キーボード/マウス	ゲームパッド
カーソル移動	マウス / 方向キー	左スティック・十字
決定	左クリック / Z	A(×)
キャンセル	右クリック / X	B(○)
ターン終了	Enter	Start
カメラ	ドラッグ / ホイール	右スティック

ゲームパッド対応は Godot の InputMap にアクションを定義しておけばキーボードと同一コードで扱える。Web exportでも Gamepad API 経由で動作する。

5. クライアント構成 (Godot)

```

res://
├── autoload/
│   ├── NetClient.gd      # WebSocket接続・送受信・再接続
│   ├── GameState.gd     # 権威状態のローカルミラー
│   └── Rules.gd         # 移動範囲・ダメージ計算などの純粋関数群
├── scenes/
│   ├── Title.tscn
│   ├── Matching.tscn
│   ├── Battle/
│   │   ├── Battle.tscn
│   │   ├── MapView.gd   # TileMapLayer描画
│   │   ├── UnitNode.gd  # 1ユニットの見た目とアニメ
│   │   ├── Cursor.gd
│   │   └── ActionMenu.gd
│   └── Result.tscn
├── data/
│   ├── units.json
│   ├── terrain.json
│   └── maps/map01.json
└── net/
    └── protocol.gd      # メッセージ定義とシリアライズ

```

設計上の要点

Rules.gd は状態を持たない純粋関数だけにする。 (state, command) -> new_state の形にしておけば、コマンドログの再生も for cmd in log: state = Rules.apply(state, cmd) の1行で済む。再接続復帰・リプレイ・デシンク検証が全部この関数の使い回しになる。

描画とロジックを分離する。 GameState は純データ。MapView / UnitNode はそれを見て描画するだけ。ログ再生時はアニメーションをスキップして状態だけ進めたいので、ここが混ざっていると再接続処理で詰む。

受信キュー方式。 commit を受け取ったらキューに積み、アニメーション再生が終わったら次を取り出す。連続でコマンドが飛んできて描画が破綻しない。

6. サーバー構成 (PartyKit)

```

party/
├── lobby.ts      # マッチメイキング用の単一Room
├── match.ts      # 1試合 = 1 Room
├── rules.ts      # 検証に必要な最小ルール (所有権・資金・手番)
└── types.ts      # Command / メッセージ型定義 (クライアントと共有したい)

```

マッチング方式

専用の lobby Roomを1つ立て、待機中の接続を保持する。2人揃ったら matchId を生成し、両者に matched を返して match Roomへ再接続させる。

```
Client → lobby Room → (2人揃う) → matchId発行  
Client → match/<matchId> Room → 試合開始
```

v1では**先着順のみ**。レーティングも部屋番号指定もなし。ただし友達と繋いでテストしたいので、**合言葉指定でのプライベートマッチだけは早めに入れておく**と開発が楽。

ターンタイマー

Durable Object の alarm API でサーバー側タイマーを実装する。setTimeout はDurable Objectがハイバネートすると消えるため使わない。

7. 取り入れると面白そうな要素

「マルチプレイヤー実装」という主題を補強するもの、かつコマンドログ方式からほぼタダで生えてくるものを優先する。

★ 採用推奨

1. リプレイ機能

コマンドログをJSONで保存 → 再生するだけ。実装コストほぼゼロで、「ネットワーク設計が正しいことの証明」としてポートフォリオ映えが最強。ブログ記事に実際のリプレイを埋め込める。

2. 観戦モード

match Roomに読み取り専用で接続させ、ログを流すだけ。「同じ設計から複数機能が導出される」ことの実例になる。

3. 遅延・パケットロスのシミュレータ

開発用に人為的な遅延を入れるトグル。デバッグに実用的で、記事では「実際にこう壊れる」のスクリーンショットが撮れる。

○ 余力があれば

4. 定型スタンプ

自由入力チャットはモデレーションが必要になるので、定型8種程度に固定。リアルタイム通信の追加チャネルとして手頃。

5. 早見え演出

相手のターン中、相手のカーソル位置をリアルタイムで送って表示する。**コマンド確定前の生の入力を流す**という点で、ターン制なのにリアルタイム同期の実装練習になる。地味だが技術的には一番おいしい。

× 今回はやらない

- CO能力 / 必殺技（ゲームバランス設計に時間を吸われる）
- 同時ターン制（面白いが、原作ルールから離れすぎ）
- マップ複数 / ランダム生成
- 索敵（3.8参照）

8. 開発マイルストーン

#	フェーズ	完了条件	目安
1	ローカル版	1画面で2人分を交互操作。ネットワークなし。ルールが完成	全体の40%
2	通信の骨組み	PartyKitに接続し、commit を往復させて2クライアントで盤面が同期	20%
3	マッチング	lobby Room経由で2人が試合開始まで到達	10%
4	堅牢化	切断復帰・ターンタイマー・デシンク検知	15%
5	画面整備	タイトル/マッチング/リザルトの3画面と演出	10%
6	追加要素	リプレイ、観戦	5%

順序の理由

フェーズ1でネットワークを一切書かないのが最重要。ルールが固まっていない状態で同期を書くと、ルール変更のたびにプロトコルとサーバー検証を直すことになり、確実に破綻する。

Rules.apply(state, cmd) の形さえ先に完成させておけば、フェーズ2は「その cmd を WebSocketに通す」だけの作業になる。

9. 想定される詰まりどころ

事前に把握しておくポイント。記事のネタにもなる。

箇所	内容	対策
Web export	GodotのWeb書き出しは初回ロードが重い（数十MB）	圧縮設定、ローディング画面必須
WebSocket	WebSocketPeer は毎フレーム poll() が必要	_process で回す。忘れると無反応になる
再接続	ログ再生中にアニメが走ると数十秒固まる	再生モードフラグで演出を全スキップ
デシンク	発生しても気づかないのが最悪	毎ターン終了時にハッシュ照合を強制
Durable Object	ハイバネートで揮発する状態がある	タイマーは alarm、状態は storage へ
型の二重管理	Command定義がGDScriptとTSに分散	JSONスキーマを単一ソースにして両方を生成、が理想（v1では手動同期で妥協）

10. ブログ記事としての切り口

技術記事にするなら、この順に書くと成立しやすい。

1. なぜGodot標準のマルチプレイAPIを使わなかったか
2. 権威モデル3案の比較と、案Bを選んだ理由
3. 「乱数をサーバーが振る」だけで決定論の問題が消える話
4. コマンドログひとつからリプレイ・観戦・再接続が全部生えた話
5. 実際に起きたデシンクと、その原因
6. 索敵を切った理由（＝隠蔽情報と権威サーバーの関係）

5番は実装が終わるまで書けないので、開発中はデシンクのログを必ず残しておくこと。これが一番読まれるパートになる。